

Coding & Solution

Date	7 July 2026
Team ID	XXXXXX
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/jahnavibezawada07/Emotion-Detection-and-Learning-support-Engine
Programming Language(s)	Python
Framework(s) Used	Streamlit, TensorFlow/Keras, HuggingFace Transformers, Plotly
Key Features Implemented	1. Deep learning-based emotion detection from text using custom BiLSTM & BERT models. 2. Integration with Google Gemini API for personalized, empathetic academic support. 3. Interactive, visual Analytics Dashboard to track emotional journey history. 4. Secure user authentication system using SQLite and bcrypt.
Pending / Incomplete Features	None. A fully functional prototype has been completely implemented. <i>(Optional: "Cloud deployment is pending; currently running flawlessly on local environment.")</i>

Setup / Run Instructions	1. Clone the repository. 2. Activate virtual environment and run: pip install -r requirements.txt 3. Create a .env file and add your GEMINI_API_KEY. 4. Run the application: python -m streamlit run app.py.
---------------------------------	---



Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The solution effectively integrates Natural Language Processing (NLP) models with Generative AI (Google Gemini) to solve the core problem statement of providing empathetic learning support. The codebase adheres strictly to PEP-8 Python standards, gracefully handles dynamic user inputs and API failures using try-except blocks, and successfully visualizes user data locally. The project is fully version-controlled on GitHub and is ready for real-world testing.
--